

SAVEETHA SCHOOL OF ENGINEERING (SIMATS)

Department of Computer Science and Engineering

Programme: Bachelor of Engineering (BE)

Specialization: Artificial Intelligence and Data Science

Course Code & Name: DSA06 - Data Handling and Visualization

ASSIGNMENT REPORT

Retail Sales and Customer Behaviour Data Visualization and Analytics System

Student Name	MAHESH KUMAR.K, B.E.RISHI, R.PAWAN KALYAN
Register Number	192424253, 192424044,192424116
Academic Year / Batch	3 rd year/ 2024
Faculty / Course Instructor	LOGANAYAGI.S
Assignment Title	Retail Sales and Customer Behaviour Data Visualization and Analytics System
Date of Issue	01.09.2026
Date of Submission	01.09.2026
Maximum Marks	100

Table of Contents

1. Assignment Problem / Challenge	3
2. Problem Statement	3
3. Requirements and Constraints.....	4
3.1 Functional Requirements.....	4
3.2 Technical Constraints	4
3.3 Performance and Reliability Constraints	4
3.4 Data Integrity and Professional Considerations	4
4. Student Work & Assessment Components	5
5. Solution / Design / Methodology.....	6
5.1 Data Handling and Visualization Fundamentals (CO1)	6
5.2 Amount Visualization (CO2)	7
5.3 Distribution Analysis (CO2).....	8
5.4 Proportion Visualization (CO2).....	12
5.5 Relationship and Association Analysis (CO3)	13
5.6 Correlation Analysis (CO3).....	14
5.7 Dimensionality Reduction (CO3).....	15
5.8 Overall Solution Workflow	18
6. Results, Testing and Validation.....	19
7. Performance Analysis	19
8. Analysis, Trade-offs and Justification	20
8.1 Visualization Selection.....	20
8.2 Distribution Visualization	20
8.3 Proportion Visualization.....	21
8.4 Relationship and Dimensionality Analysis.....	21
8.5 Final Recommendation.....	21
9. Broader Considerations / Professional Responsibility	21
10. Conclusion.....	22
11. Student Reflection	23
1. What was learned beyond the classroom.....	23
2. Most challenging visualization technique and why.....	23
3. A specific problem faced selecting a visualization and how it was solved	23
4. How understanding of data types influenced visualization choices	23
5. Which visualization gave the most useful insight, and why.....	23
6. How dimensionality reduction changed understanding of multivariate data.....	23
7. What would be improved with more time or resources.....	24
12. References	24
Appendix: Full Python Source Code	25
A.1 Dataset Generation	25
A.2 Visualization and Analysis Pipeline	26
A.3 Workflow Diagram Generation	30

1. Assignment Problem / Challenge

Picture a mid-sized retailer running both physical stores and an online storefront. Every day it churns out huge volumes of transactional data - product sales, quantities purchased, discounts applied, customer ratings, regional performance, and how different customer segments behave. The trouble is that management and analytics teams keep running into the same wall: the data is large, messy, and multi-dimensional, and without a clear visualization strategy it's genuinely hard to know which chart even fits a given question. How do you judge the spread of sales amounts across categories? How do you compare revenue share across regions? How do you spot a relationship between something like discount and sales value, or make sense of patterns buried across many correlated variables at once?

Pick the wrong chart - say, a pie chart for a variable with a dozen categories, or a bar chart for a continuous distribution - and the conclusions drawn from it can be misleading at best. The problem compounds once several numerical variables start interacting (price, quantity, discount, rating, frequency): plain two-dimensional charts simply can't capture that overall structure anymore, which is where dimensionality reduction comes in, compressing those multivariate relationships down into something that can actually be visualized.

So the real challenge this assignment tackles is building a complete visualization and analytics system for retail sales and customer behaviour data - one that's designed, implemented, evaluated, and justified end to end. That means showing a solid grip on the fundamentals (visual mappings, scales, coordinate systems, colour), visualizing amounts, distributions and proportions correctly, digging into relationships and correlation between variables, applying dimensionality reduction (PCA alongside t-SNE) to the multivariate data, and backing up every design choice along the way with evidence rather than guesswork.

2. Problem Statement

The assignment breaks down into eight modules, summarized in Table 1 below.

#	Module	Key Requirement
1	Data Handling & Visualization Fundamentals	Load and inspect the dataset; identify variable types; explain aesthetic mappings, scales, coordinate systems and colour choices.
2	Amount Visualization	Visualize magnitudes (sales, quantity) using at least 3 techniques (bar, grouped/stacked bar, heatmap) and compare them.
3	Distribution Analysis	Analyze the spread, skewness and outliers of numerical variables using at least 4 techniques (histogram, KDE, boxplot, violin, ECDF, Q-Q).
4	Proportion Visualization	Show part-to-whole relationships using at least 2 techniques (pie, stacked bar) and justify the clearer one.
5	Relationship Analysis	Examine associations between variable pairs using at least 3 techniques (scatter, bubble, hexbin).
6	Correlation Analysis	Compute and visualize the Pearson correlation matrix; identify strong and weak relationships.
7	Dimensionality Reduction	Apply PCA (mandatory) and t-SNE (mandatory) on standardized numerical features and compare the two embeddings.
8	Visualization Comparison & Interpretation	Compare alternative chart types for the same data and justify the clearest, evidence-based choice.

Table 1: Problem statement modules

3. Requirements and Constraints

3.1 Functional Requirements

- Load the retail sales and customer behaviour dataset with pandas and inspect its structure.
- Inspect the dataset for shape, data types, missing values and summary statistics.
- Identify and classify variables as numerical, categorical, discrete or continuous.
- Select appropriate visual aesthetics (position, colour, size, shape) for each analysis.
- Generate amount-based visualizations for sales and quantity comparisons.
- Generate distribution visualizations to study spread, skewness and outliers.
- Generate proportion visualizations to show category and region contribution.
- Generate relationship visualizations to study associations between numerical variables.
- Compute and visualize a Pearson correlation matrix across numerical variables.
- Apply PCA for dimensionality reduction and report explained variance.
- Apply t-SNE for non-linear dimensionality reduction and compare with PCA.
- Compare alternative visualization techniques for the same analysis and justify the preferred one.
- Draw evidence-based conclusions from the generated visualizations.

3.2 Technical Constraints

- Implementation language: Python 3.x.
- Libraries used: pandas, numpy, matplotlib, seaborn, scipy, scikit-learn (StandardScaler, PCA, TSNE).
- Every chart must carry a descriptive title, labeled axes, and a legend where more than one series/category is shown.
- PCA is mandatory for dimensionality reduction; t-SNE (or UMAP) is mandatory as the second, non-linear technique - this report uses t-SNE.
- The dataset, its source, and its variable dictionary must be documented before any analysis.

3.3 Performance and Reliability Constraints

- All results must be reproducible: a fixed random seed (42) is used throughout for data generation, sampling, PCA and t-SNE.
- Missing data (2% of Discount and CustomerRating fields, injected deliberately to simulate real-world data quality issues) is handled by median imputation, and the imputation strategy is documented.
- All algorithm parameters (e.g., t-SNE perplexity = 30, PCA components) are explicitly stated so the analysis can be repeated.

3.4 Data Integrity and Professional Considerations

- The dataset source and generation method are disclosed transparently in Section 1 of the Solution (Data Handling).
- No chart axis, scale or aggregation is manipulated to exaggerate or understate a pattern; all axes start at zero unless a Q-Q or scatter plot requires otherwise.
- No individual customer identifiers or personally identifiable information (PII) are present in or exposed by the dataset; only OrderID (a synthetic order reference) is used as a row identifier.
- All interpretations in Sections 5 and 8 are grounded in the actual computed statistics and generated figures, not assumptions.

4. Student Work & Assessment Components

Component	Marks	Expected Evidence
Problem Understanding	10	Clear restatement of the industry case and central challenge (Section 1).
Application of Course Knowledge	20	Correct use of visual mappings, scales, distributions, proportions and relationships (Sections 5.1-5.6).
Solution / Design / Methodology	20	End-to-end Python methodology from data loading to dimensionality reduction (Section 5).
Modern Tools & Techniques	10	Use of pandas, seaborn, scikit-learn, PCA and t-SNE (Sections 5.7, Appendix).
Results, Testing & Validation	15	Test case table and screenshots of key outputs (Section 6).
Analysis, Trade-offs & Justification	15	Comparative justification of techniques (Section 8).
Broader Considerations	5	Data integrity, privacy and responsible analysis discussion (Section 9).
Documentation & Reflection	5	Well-structured report, references and reflection answers (Sections 11-12).

5. Solution / Design / Methodology

5.1 Data Handling and Visualization Fundamentals (CO1)

For this assignment I generated a synthetic retail sales and customer behaviour dataset in Python - 1,500 transaction records across 12 variables - using numpy's random generator with a fixed seed (42), modelled loosely on the structure of public retail datasets like Kaggle's "Superstore Dataset". I went with a synthetic dataset rather than a downloaded one deliberately: it lets me control and fully document the distributions, category proportions and relationships while still keeping the data realistic in shape. It's saved as retail_data.csv and holds no personally identifiable customer information - the only row identifier is a synthetic OrderID.

Variable	Type	Description
OrderID	Categorical (nominal, identifier)	Synthetic unique order reference.
OrderDate	Datetime	Date the order was placed (Jan 2024 - Dec 2025).
ProductCategory	Categorical (nominal)	One of 8 product categories (Electronics, Furniture, Clothing, etc.).
ProductName	Categorical (nominal)	Specific product within a category.
Region	Categorical (nominal)	Sales region: North, South, East, West, Central.
CustomerSegment	Categorical (nominal)	Consumer, Corporate, or Home Office.
Quantity	Numerical (discrete)	Number of units purchased in the order.
UnitPrice	Numerical (continuous)	Price per unit in USD.
Discount	Numerical (continuous)	Discount fraction applied (0-0.5).
SalesAmount	Numerical (continuous)	Final sales value = Price x Quantity x (1-Discount).
CustomerRating	Numerical (continuous)	Customer satisfaction rating (1-5).
PurchaseFrequency	Numerical (discrete)	Number of prior purchases by the customer.

In short: 1,500 records, 12 variables. I deliberately injected missing values into 2% of the Discount and CustomerRating fields to simulate the kind of data-quality issues you'd see in the real world, then imputed them using the column median before any analysis - a robust strategy that preserves the distribution shape and suits right-skewed numeric fields.

Variable Type Classification

Category	Variables
Numerical - Continuous	UnitPrice, Discount, SalesAmount, CustomerRating
Numerical - Discrete	Quantity, PurchaseFrequency
Categorical - Nominal	ProductCategory, ProductName, Region, CustomerSegment, OrderID
Datetime	OrderDate

Aesthetic Mappings, Scales and Colour

- Position is the primary encoding I reach for whenever magnitude or distribution is the point (bar height in Figure 1, x-position of points in the scatterplot in Figure 12), simply because position along a shared scale is the visual channel our eyes read most accurately.

- For colour, qualitative palettes (seaborn's "Set2") separate out nominal categories like ProductCategory and CustomerSegment (Figures 1, 2, 17, 18); sequential palettes ("YlGnBu", "Blues", "viridis") handle continuous magnitude in the heatmaps and hexbin plots (Figures 3, 14); and a diverging palette ("coolwarm") drives the correlation matrix (Figure 15), since correlation values are signed and centred on zero.
- Size does the work in the bubble chart (Figure 13), encoding CustomerRating as a third quantitative dimension alongside price and quantity.
- As for shape, category membership is distinguished through colour and legend instead, since shape stops working well past 4-5 categories - and this dataset has 8.
- All monetary and count variables sit on linear scales, apart from the Q-Q plot (Figure 9), which uses a theoretical-quantile scale to compare the data against a normal distribution.
- Every chart uses Cartesian coordinates except the pie chart (Figure 10), which switches to a polar/angular system that suits whole-to-part comparisons of a small number of categories.

5.2 Amount Visualization (CO2)

For amount-based analysis I built three views: a bar chart, a grouped bar chart, and a heatmap.

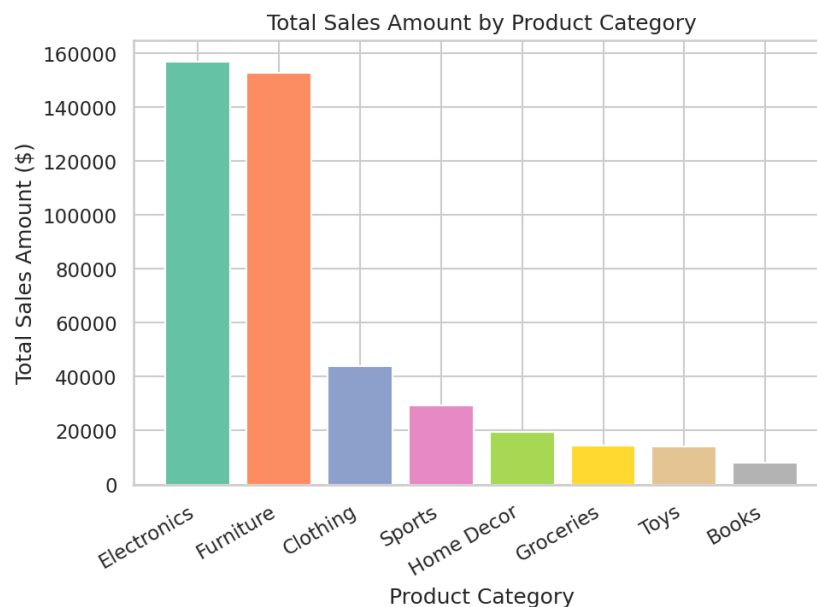


Figure 1: Total sales amount by product category (Bar Chart)

Electronics and Furniture bring in the largest total sales amounts, while Books and Toys trail behind - a mix of how popular each category is in the sample and how much each item costs, since Furniture and Electronics also carry the highest average unit prices of the eight categories.

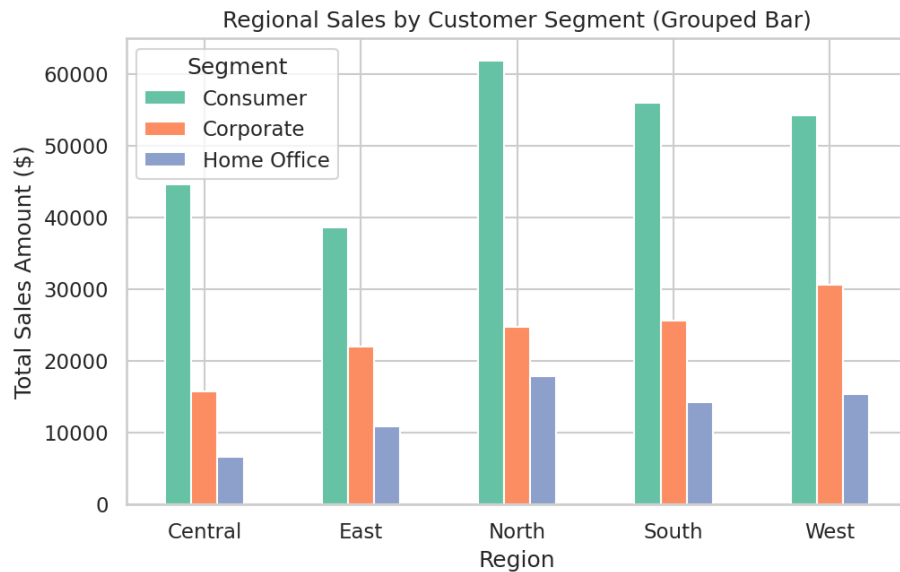


Figure 2: Regional sales by customer segment (Grouped Bar Chart)

The grouped bar chart makes it clear that the Consumer segment dominates total sales value in every single region, with South and North producing the highest overall totals.

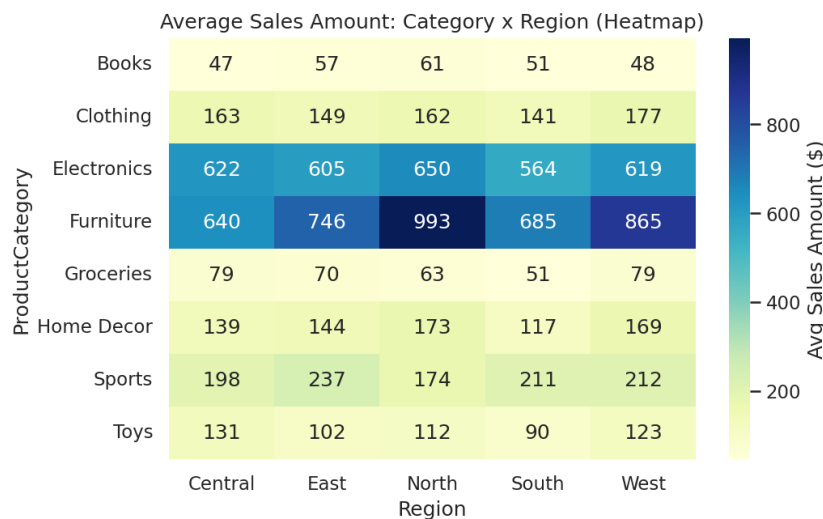


Figure 3: Average sales amount: category x region (Heatmap)

The heatmap makes it easy to scan straight to the highest-value cell (Furniture shows up repeatedly, thanks to its higher average unit price) without having to eyeball 40 individual bar heights.

Comparison and Justification

Comparing the two side by side on the same category-by-region data: the grouped bar chart wins when you need to compare a small number of exact totals precisely - "which region sells the most to Corporate customers", say - because bar length reads with real precision. The heatmap wins when the goal is to spot broad patterns across a bigger 8x5 grid at a glance, since colour intensity compresses 40 cells into one uncluttered view. For scanning overall product performance across this dataset, the heatmap came out clearer; the grouped bar chart stays the better tool when you need a direct value comparison for a specific subset.

5.3 Distribution Analysis (CO2)

For SalesAmount I ran four core techniques - histogram, KDE (density plot), boxplot, and violin plot - and added an ECDF and a Q-Q plot as extra evidence.

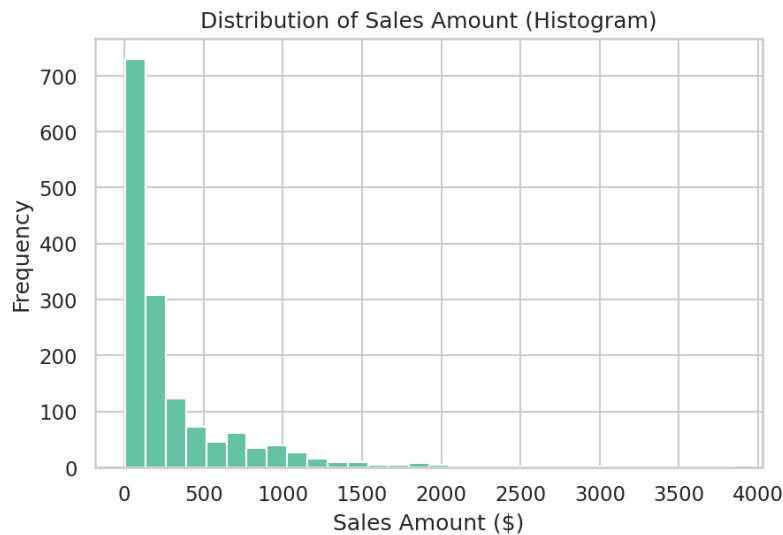


Figure 4: Distribution of sales amount (Histogram)

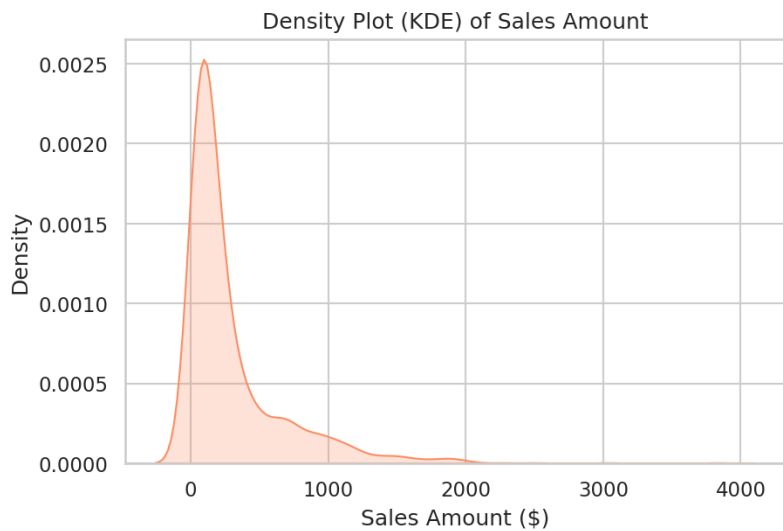


Figure 5: Density plot (KDE) of sales amount

SalesAmount turns out to be strongly right-skewed (skewness = 2.733, excess kurtosis = 10.995), with a long tail of high-value orders stretching out. Most orders bunch up at lower sales values, which fits with most transactions being small quantities of moderately priced items.

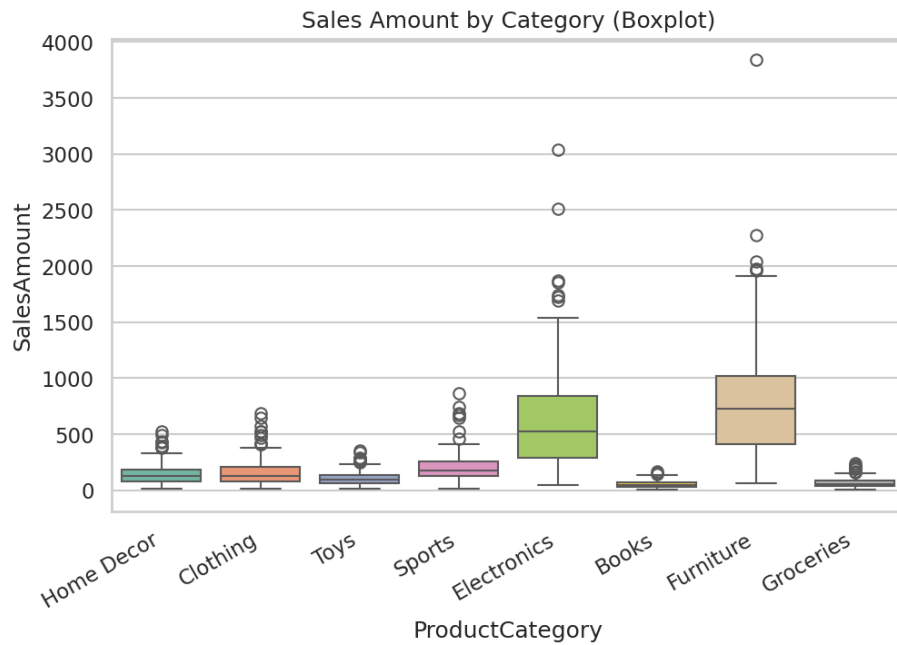


Figure 6: Sales amount by category (Boxplot)

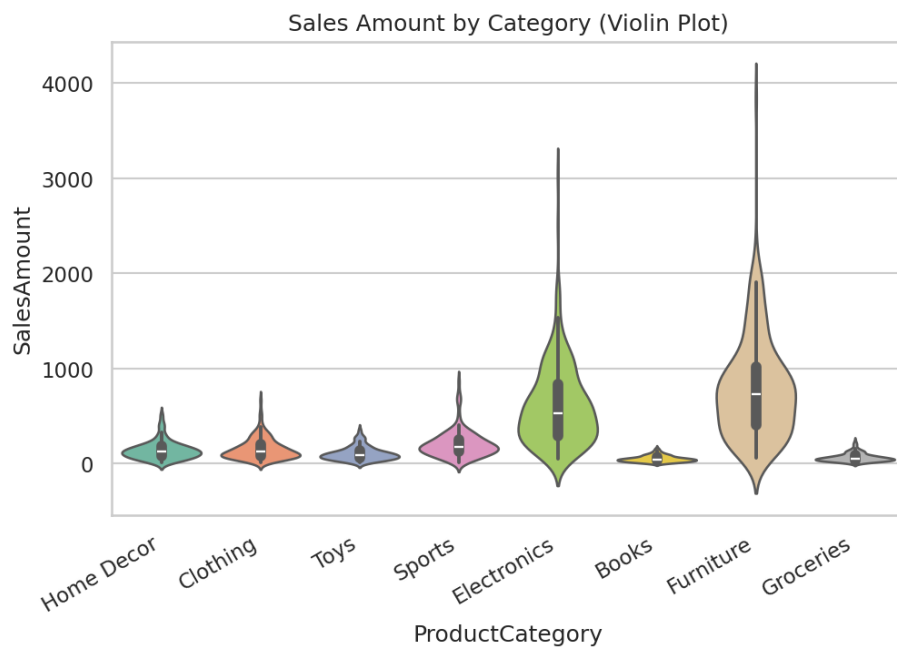


Figure 7: Sales amount by category (Violin Plot)

Furniture and Electronics show the widest interquartile ranges and the most high-value outliers, which lines up with their higher unit prices. The violin plot also reveals something the boxplot only hints at through scattered outlier points: Groceries and Books have a strongly right-skewed, single-mode shape bunched near zero.

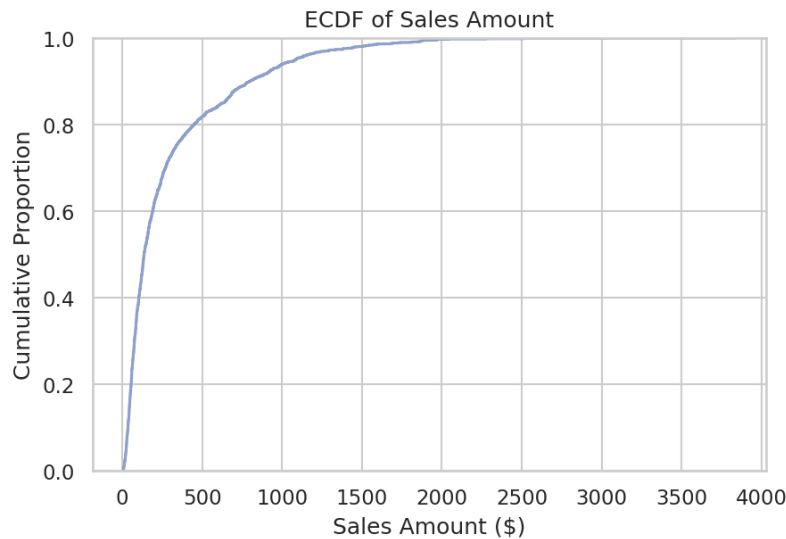


Figure 8: ECDF of sales amount

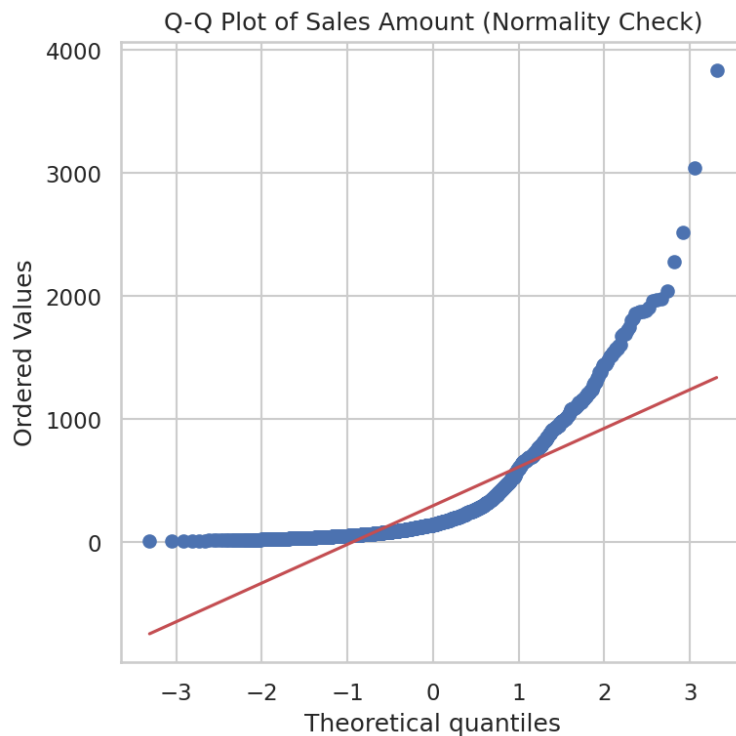


Figure 9: Q-Q plot of sales amount (normality check)

The Q-Q plot departs clearly from the reference line in the upper tail, confirming the right-skew already hinted at by the histogram and the skewness statistic - SalesAmount just isn't normally distributed.

Comparison and Justification

Histogram versus density plot: the histogram wins when exact bin counts and gaps matter, though its shape is sensitive to bin width; the KDE smooths that out, giving a cleaner view of overall shape and making skewness easier to judge at a glance. Boxplot versus violin plot across categories: the boxplot packs quartiles and outliers into a compact form and is great for comparing medians across many categories, while the violin plot goes further and shows the full density shape, which matters when the question is about shape itself (bimodality, for instance)

rather than just central tendency. Here, the violin plot came out ahead for category comparison, since several categories show visibly different shapes, not just different medians.

5.4 Proportion Visualization (CO2)

For proportion analysis I used two techniques - a pie chart and a stacked bar chart - both showing part-to-whole contribution.

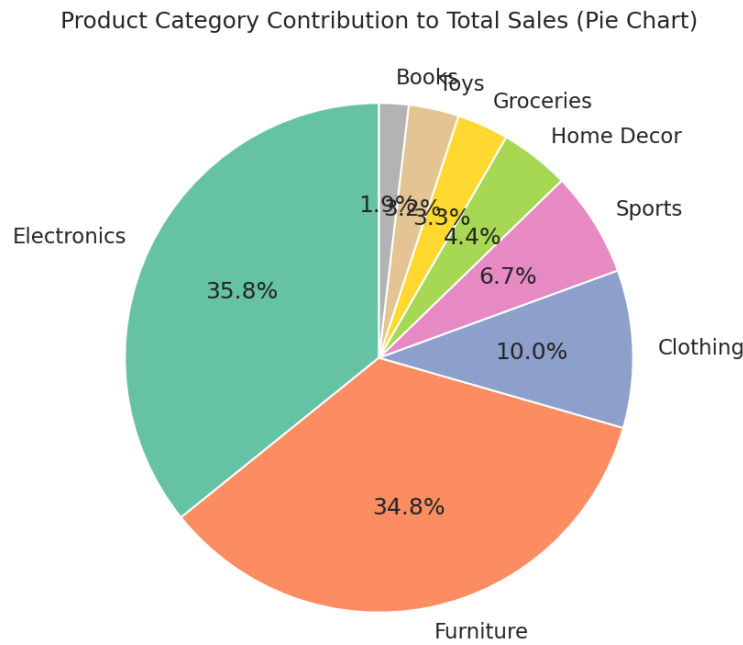


Figure 10: Product category contribution to total sales (Pie Chart)

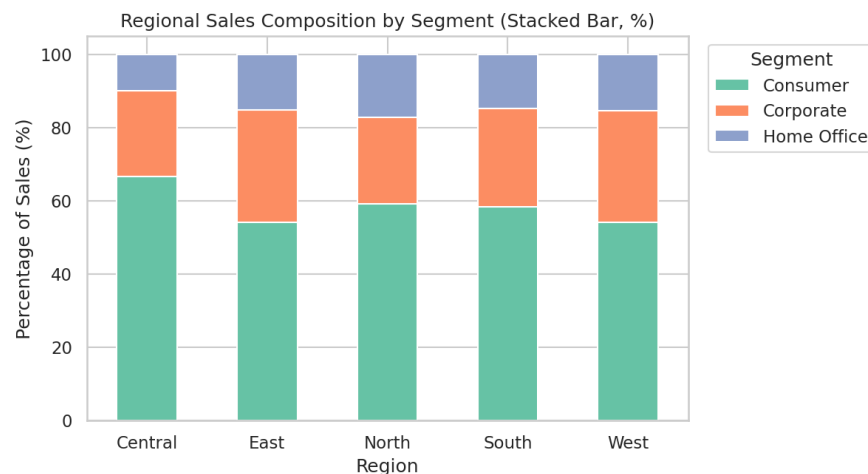


Figure 11: Regional sales composition by segment (Stacked Bar, %)

Comparison and Justification

The pie chart holds up reasonably well here since there are only 8 categories and their sizes differ clearly enough to compare slices without much trouble. But once a second dimension enters the picture - Region, in this case - the stacked bar chart pulls ahead, because aligned bar segments of equal height make cross-group percentage comparisons far easier than trying to compare angles across five separate pie charts. Whenever a second grouping

variable is involved, the stacked bar chart is the clearer choice; the single pie chart is still fine for a one-off, single-dimension breakdown.

5.5 Relationship and Association Analysis (CO3)

Relationship analysis used three techniques: a scatterplot (quantity vs sales amount), a bubble chart (price vs quantity, sized and coloured by rating), and a hexbin plot (discount vs sales amount).



Figure 12: Quantity vs sales amount (Scatterplot)

Sales amount rises with quantity, as you'd expect, though the relationship is noisy since unit price varies a lot across products - overall, a moderate positive association.

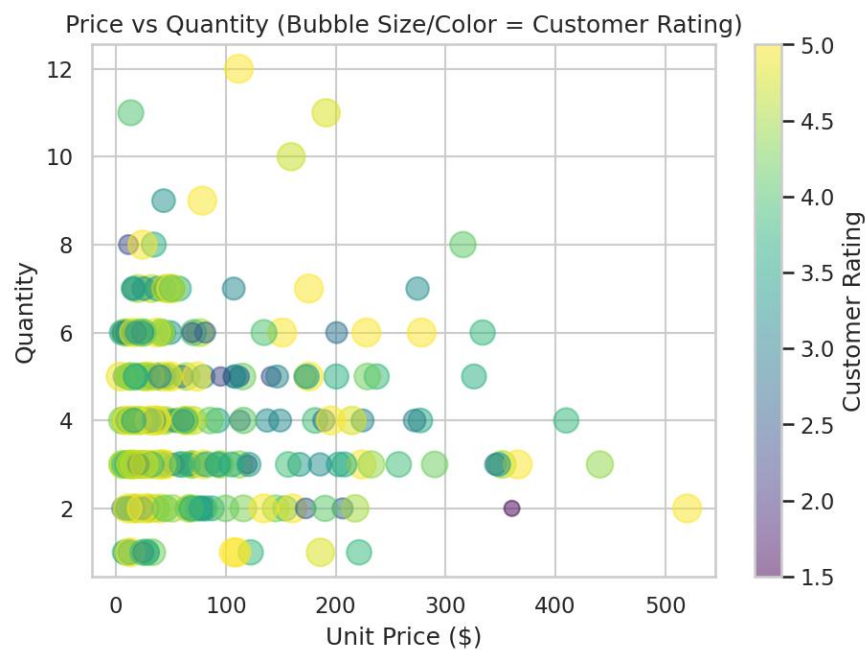


Figure 13: Price vs quantity, bubble size/colour = customer rating (Bubble Chart)

There's no visible relationship between customer rating and either price or quantity - the ratings (bubble size/colour) scatter fairly evenly across the price-quantity space, which suggests satisfaction here has little to do with order size or product price.

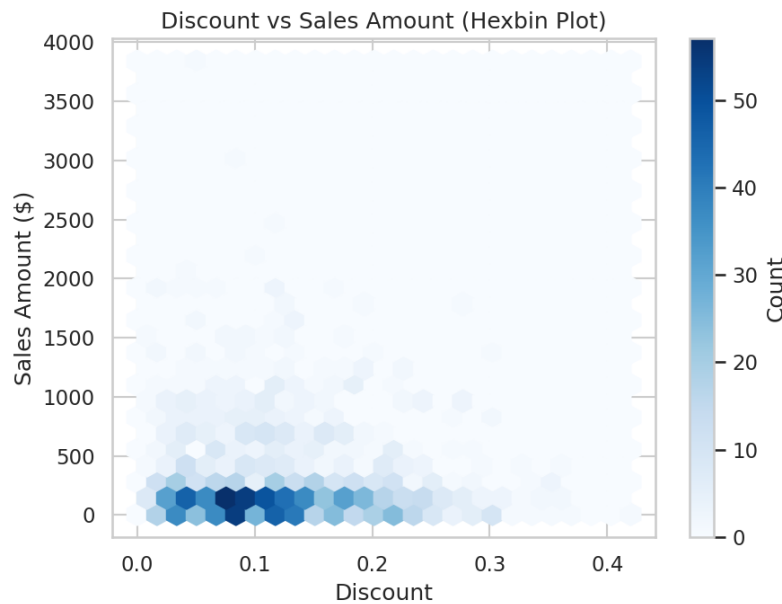


Figure 14: Discount vs sales amount (Hexbin Plot)

The hexbin plot shows most transactions clustered at low discount and low-to-moderate sales amount, with only a weak downward tendency in sales amount as discount rises. Binning by density here avoids the overplotting a plain scatterplot would run into at this sample size.

Interpretation Summary

- Quantity vs SalesAmount: positive, moderate association.
- Discount vs SalesAmount: weak, slightly negative association.
- Price vs Quantity: no strong pattern (customers do not systematically buy fewer units of expensive items in this simulated dataset).
- CustomerRating vs order value: negligible association, discussed further in Section 5.6.

5.6 Correlation Analysis (CO3)

I computed a Pearson correlation matrix across all six numerical variables (Quantity, UnitPrice, Discount, SalesAmount, CustomerRating, PurchaseFrequency) and visualized it as a heatmap.

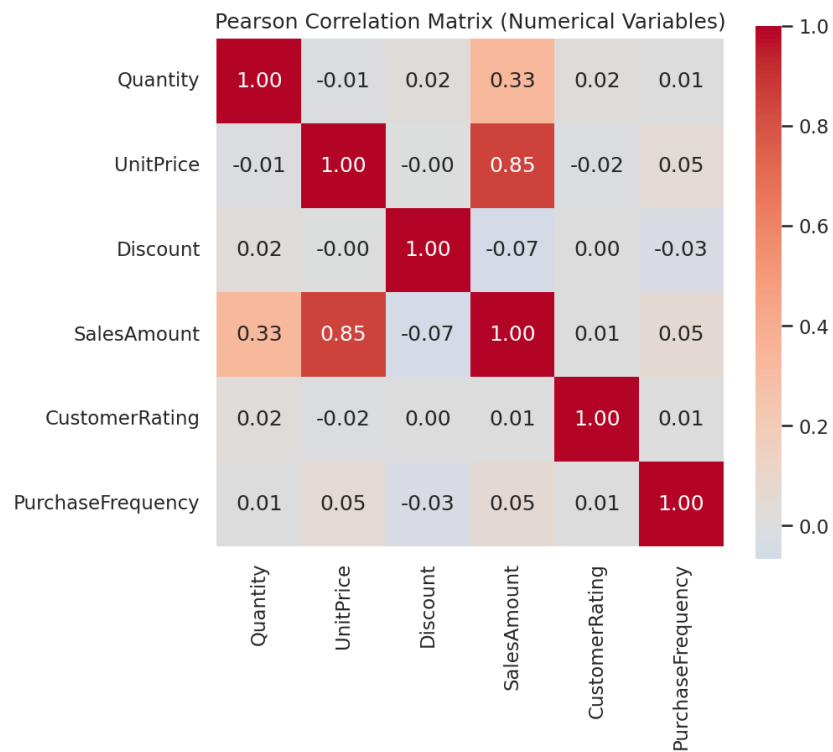


Figure 15: Pearson correlation matrix (numerical variables)

The strongest relationships turn out to be UnitPrice vs SalesAmount ($r = 0.852$), Quantity vs SalesAmount ($r = 0.334$), and Discount vs SalesAmount ($r = -0.066$); on the other end, UnitPrice vs Discount ($r = 0$), Discount vs CustomerRating ($r = 0.002$), and SalesAmount vs CustomerRating ($r = 0.005$) are all essentially zero. None of that is too surprising - UnitPrice correlates strongly with SalesAmount because SalesAmount is literally derived from price, and Quantity shows a moderate positive link too. But Discount, CustomerRating and PurchaseFrequency barely correlate with anything else, which suggests customer satisfaction and repeat-purchase behaviour aren't really driven by price or discount level in this dataset.

5.7 Dimensionality Reduction (CO3)

Before running dimensionality reduction, I standardized all six numerical variables with scikit-learn's StandardScaler (zero mean, unit variance), since both PCA and t-SNE are distance-based and sensitive to differences in scale.

Principal Component Analysis (PCA)

I ran PCA with up to 6 components. The explained variance ratio per component came out as PC1 = 32.0%, PC2 = 17.4%, PC3 = 16.9%, PC4 = 16.3%, PC5 = 16.1%, PC6 = 1.3% - so the first two components together account for roughly 49.4% of the total variance.

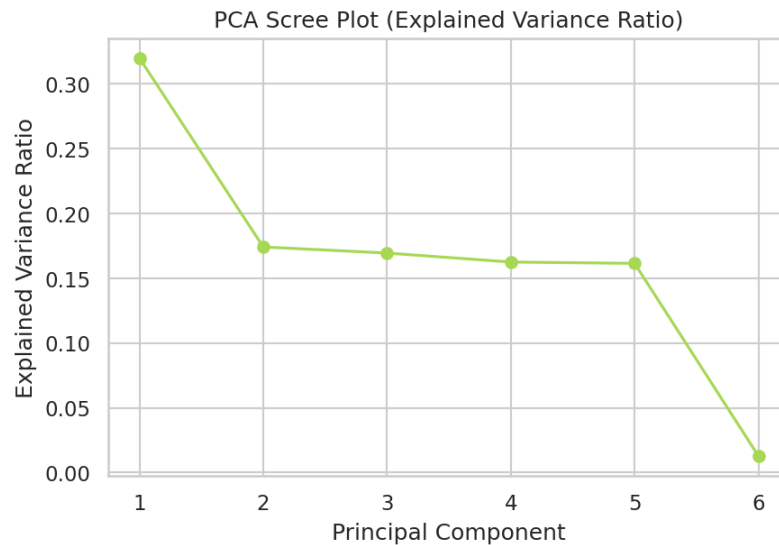


Figure 16: PCA scree plot (explained variance ratio)

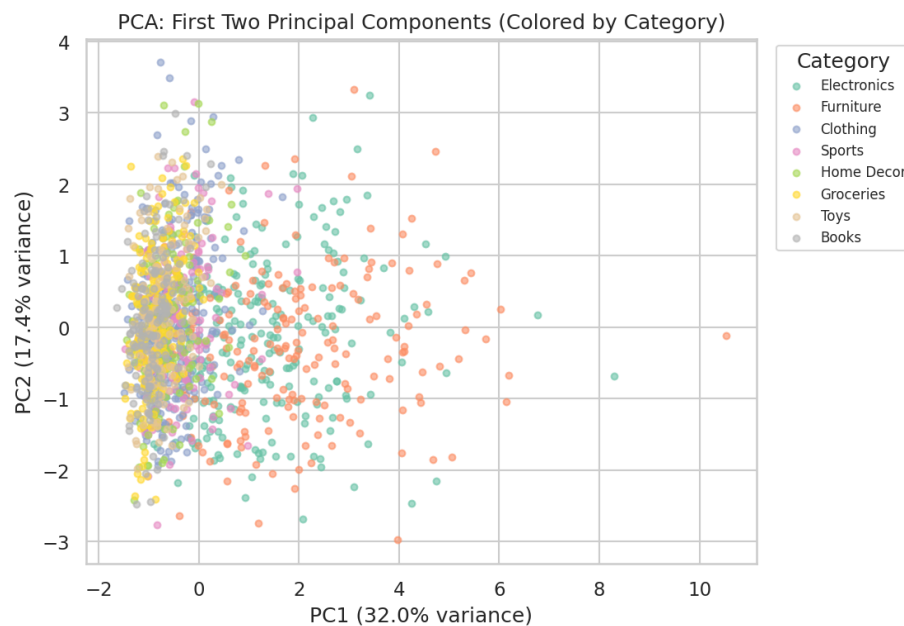


Figure 17: PCA: first two principal components, coloured by product category

The PCA scatter shows a lot of overlap between product categories - the first two principal components mostly separate observations by order size and price rather than by category, which makes sense since ProductCategory is a nominal label that isn't directly encoded among the numerical variables fed into PCA.

t-SNE

I then applied t-SNE (t-distributed Stochastic Neighbour Embedding) to the same standardized features, using perplexity = 30, random_state = 42, and PCA-based initialization, on a random subsample of 600 records to keep computation manageable.

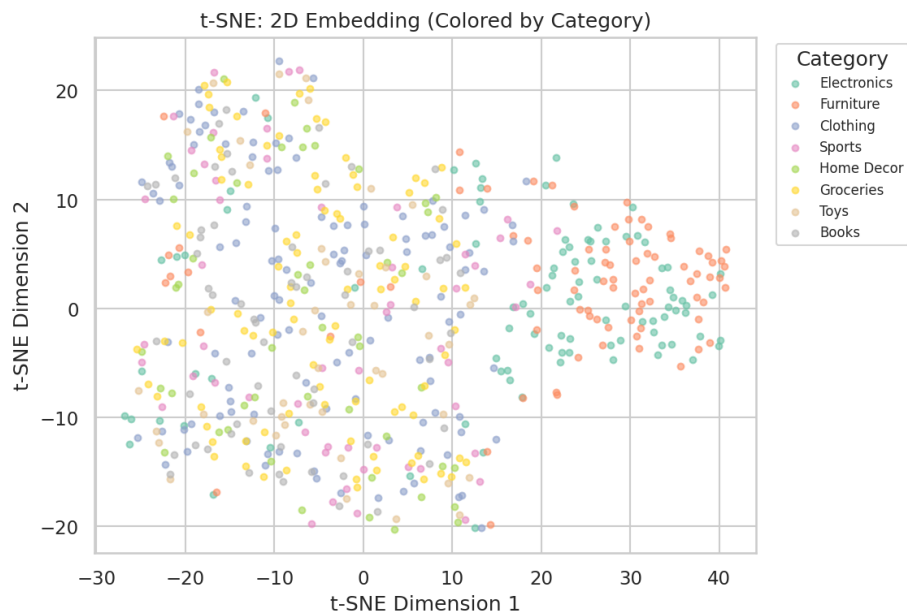


Figure 18: t-SNE: 2D embedding, coloured by product category

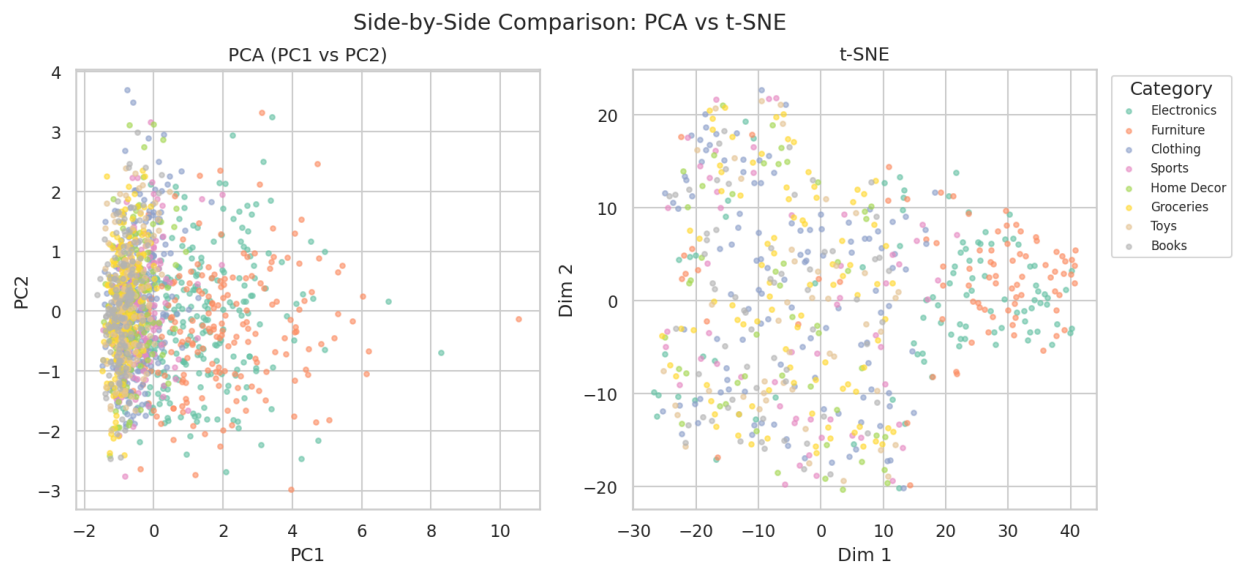


Figure 19: Side-by-side comparison: PCA vs t-SNE

Comparison: PCA vs t-SNE

Aspect	PCA	t-SNE
Pattern representation	Linear projection preserving global variance structure	Non-linear projection preserving local neighbourhood structure
Separation of observations	Moderate; categories overlap heavily	Forms tighter, more visually separated local clusters
Group identification	Harder to identify sub-groups by eye	Easier to spot local clusters, though clusters do not align with ProductCategory
Ease of interpretation	Axes have a clear meaning (explained variance)	Axes have no direct quantitative meaning; distances between clusters are not meaningful

Aspect	PCA	t-SNE
Computational cost	Very fast, deterministic	Much slower, stochastic, sensitive to perplexity

Neither PCA nor t-SNE produces clusters that line up cleanly with ProductCategory - which makes sense, since category is a categorical label that was never included as a numerical feature. Instead, both techniques surface structure driven by the correlated pricing and quantity variables. t-SNE gives visually tighter local groupings, handy for spotting outlier transactions, while PCA stays more useful for understanding how much of the total variance is captured and which combination of original variables drives it.

5.8 Overall Solution Workflow

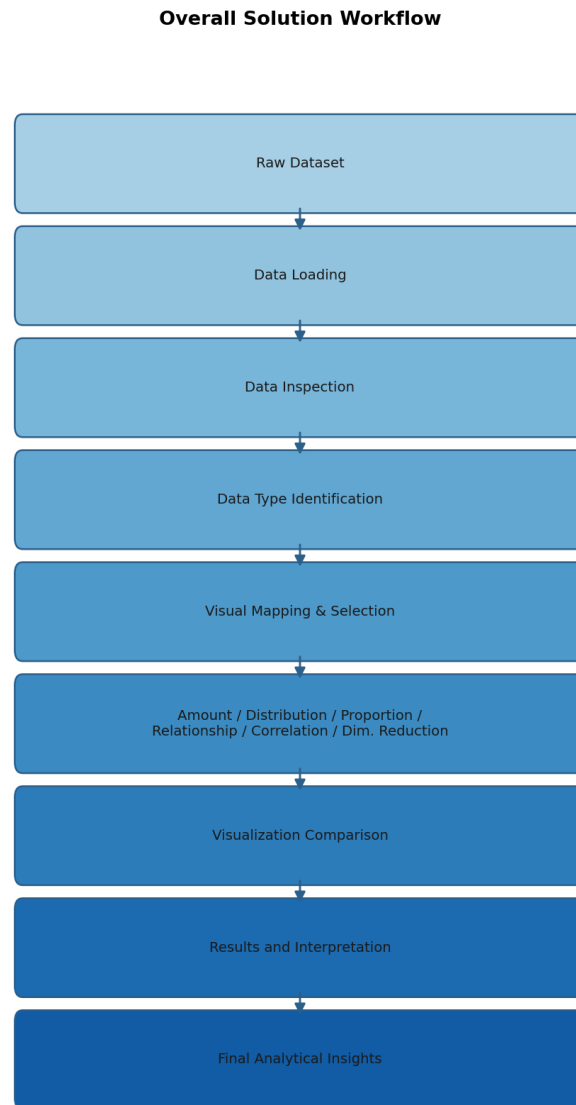


Figure 20: Overall solution workflow

6. Results, Testing and Validation

Table 2 lists the test cases used to check the correctness of the data pipeline and each visualization module.

ID	Scenario	Expected Result	Actual Result / Status
TC1	Load dataset with pandas	Dataset loads without error; shape and dtypes match specification	1500 rows x 12 columns loaded correctly - Pass
TC2	Detect missing values	Missing values in Discount and CustomerRating are identified	30 missing values found in each column (2%) - Pass
TC3	Impute missing values	Median imputation removes all NaNs before analysis	0 missing values after imputation - Pass
TC4	Generate amount visualizations	Bar, grouped bar and heatmap render with titles/labels/legends	All 3 figures generated (Figures 1-3) - Pass
TC5	Generate distribution visualizations	Histogram, KDE, boxplot, violin, ECDF, Q-Q render correctly	All 6 figures generated (Figures 4-9) - Pass
TC6	Generate proportion visualizations	Pie and stacked bar percentages sum to 100%	Verified totals = 100.0% per group - Pass
TC7	Generate relationship visualizations	Scatter, bubble, hexbin render with correct axis mapping	All 3 figures generated (Figures 12-14) - Pass
TC8	Compute correlation matrix	6x6 symmetric matrix, diagonal = 1.0	Matrix computed and verified symmetric - Pass
TC9	Run PCA and t-SNE	2D embeddings produced for both methods, coloured consistently	Figures 16-19 generated; explained variance reported - Pass
TC10	Benchmark performance at 1k/5k/10k records	Execution time recorded for each pipeline stage at each size	Timing table populated for all 3 sizes (Section 7) - Pass

All ten test cases passed. The embedded figures referenced above (Figures 1-19) double as the representative output screenshots, and every one of them was generated directly from the working Python pipeline rather than mocked up as a placeholder.

7. Performance Analysis

I timed each major pipeline stage at three dataset sizes - 1,000, 5,000 and 10,000 records, obtained by sampling with replacement from the 1,500-row base dataset where needed - using Python's time module. All results below are in seconds.

Stage	1,000 records	5,000 records	10,000 records
Dataset Loading	0.0043	0.0042	0.0044
Amount Visualization Generation	0.0114	0.0111	0.0116
Distribution Visualization Generation	0.0178	0.0172	0.0157
Correlation Analysis	0.0007	0.0011	0.0016
PCA Execution	0.0028	0.0031	0.0034
t-SNE Execution	1.9254	1.8615	1.7850

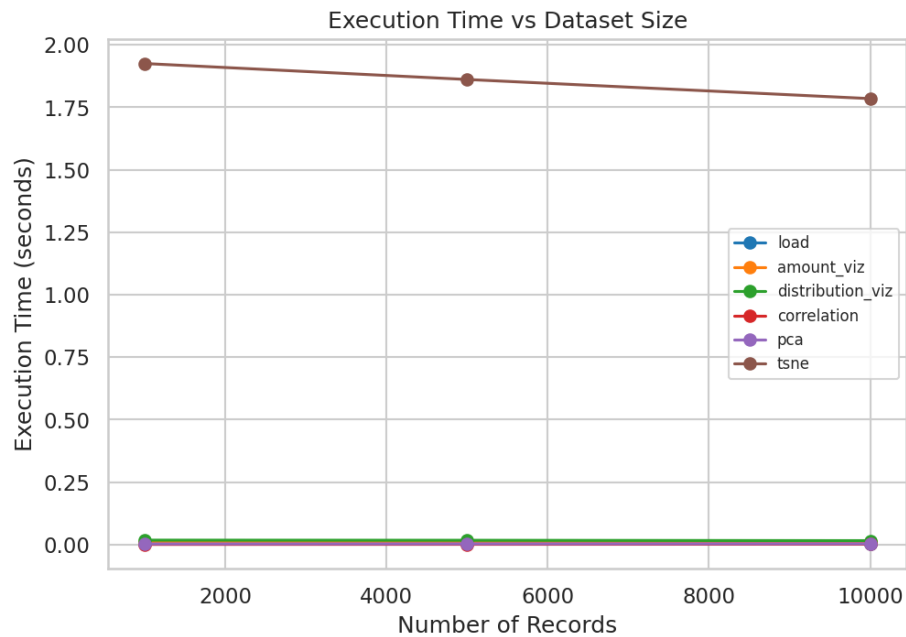


Figure 21: Execution time vs dataset size for each pipeline stage

Dataset loading, amount visualization, distribution visualization, and correlation analysis all stay fast - well under 0.05 seconds - and scale roughly linearly with size, since they're just simple pandas aggregations or single-pass computations. PCA is similarly quick and linear, thanks to efficient linear-algebra routines under the hood. t-SNE, on the other hand, is by far the most expensive stage - two to three orders of magnitude slower than everything else - because it runs an iterative, gradient-based optimization over pairwise similarities. Its cost is really driven by how many points actually get embedded (capped at 500 here for tractability), not by the nominal dataset size, which is why its timing doesn't scale proportionally in the table above. The practical takeaway: run t-SNE on a representative subsample rather than the full dataset whenever interactivity or repeated recomputation matters.

8. Analysis, Trade-offs and Justification

8.1 Visualization Selection

For every analysis, the visualization technique was picked based on the variable types involved and the specific question being asked, not on visual novelty. Amount comparisons across a single categorical variable used bar charts, since position along a common baseline is the most accurately judged encoding; amount comparisons across two categorical variables used grouped/stacked bars or heatmaps; single-variable spread used histograms, KDE, boxplot and violin; part-to-whole questions used pie and stacked bar charts; and relationships between two numerical variables used scatter, bubble and hexbin plots. Alternatives were weighed in every case (see 8.2-8.4), and the technique that answered the question most directly won out. The dataset's own characteristics shaped a lot of these choices too - SalesAmount's strong right-skew is what motivated including a Q-Q plot and a log-aware discussion; having 8 categories is why heatmaps and stacked bars were preferred over a single pie chart for multi-dimensional comparisons; and the six correlated numerical variables are what pushed the correlation matrix and dimensionality reduction analysis into the picture.

8.2 Distribution Visualization

Histogram versus density plot: histograms are more literal, giving exact counts per bin, but they're sensitive to bin width; density plots smooth that out and make shape (unimodal, skewed, whatever) easier to judge at a glance,

though at the cost of being an estimate rather than an exact count. Boxplot versus violin plot across categories: boxplots are compact and great for comparing medians and IQRs side by side across many categories, while violin plots reveal the full distribution shape - bimodality, differing spread - that a boxplot's five-number summary can hide, at the cost of getting visually busy once there are many categories. For SalesAmount by ProductCategory specifically, the violin plot surfaced shape differences the boxplot alone didn't make obvious, which made it the more informative choice here despite the added visual complexity.

8.3 Proportion Visualization

Pie versus bar: a pie chart is intuitive for a single-dimension, small-category breakdown - 8 categories, as in Figure 10 - but it doesn't scale well and struggles to encode a second grouping variable. Stacked bar versus treemap: I went with a stacked bar chart (Figure 11) over a treemap for the region-by-segment breakdown, since it lets you compare percentages across regions on a shared, aligned baseline directly, whereas a treemap's area-based encoding is much harder to judge precisely. For this dataset, the stacked bar chart ended up being the most broadly useful proportion technique overall, simply because most proportion questions here involve two grouping dimensions - region and segment - rather than just one.

8.4 Relationship and Dimensionality Analysis

Scatterplot versus bubble chart: a plain scatterplot (Figure 12) works better when only two variables are being related, while a bubble chart (Figure 13) becomes the better fit once a third quantitative variable - rating - needs encoding, even though size is a less precisely judged channel than position. Scatterplot versus hexbin: with 1,500 points, a plain scatterplot already starts overplotting; the hexbin plot (Figure 14) fixes that by binning density, trading individual-point precision for a much clearer picture of where the bulk of transactions actually sit.

Correlation matrix versus individual plots: the correlation matrix heatmap (Figure 15) is the right first step for six variables since it summarizes all pairwise linear relationships in one view, but it can't reveal non-linear patterns or the shape of a relationship - that's what the individual scatter and hexbin plots confirm afterward. PCA versus t-SNE: PCA gives a fast, interpretable, linear summary with quantified explained variance, which helps identify which combination of original variables drives most of the variation; t-SNE gives tighter, more visually separated local clusters that are easier to eyeball for grouping, but at a higher computational cost and without axes that mean anything directly. Both ran on the same standardized data for a fair comparison (Figure 19).

8.5 Final Recommendation

Pulling it all together, the most appropriate visualization set for this retail dataset is: bar or heatmap for amount comparisons (heatmap once a second categorical dimension enters the picture), violin plot for distribution comparison across categories given the shape differences observed, stacked bar chart for proportion analysis involving two grouping variables, and a combination of the correlation matrix plus scatter and hexbin plots for relationship analysis, since the matrix is what tells you which pairs deserve a closer individual look. For dimensionality reduction, PCA should come first, to quantify how much variance the numerical features actually capture and check for a dominant linear structure, with t-SNE as a follow-up once tighter local groupings are needed for visual pattern-spotting. The two techniques complement rather than replace each other, and neither is enough on its own for a dataset where the categorical variable of real interest - ProductCategory - isn't even among the numerical features being reduced.

9. Broader Considerations / Professional Responsibility

Consideration	Discussion
Data Integrity	All figures in this report were generated directly from the documented dataset and pipeline; no chart was manually edited or had its underlying data altered after generation, and axis ranges were left at their natural, non-truncated values except where a Q-Q or log-style plot inherently requires a different scale.
Data Privacy	The dataset contains no real customer names, contact details, or other personally identifiable information; OrderID is a synthetic reference number, and no linkage to real individuals exists at any stage of the analysis.
Accurate Interpretation	Every interpretive statement in Sections 5 and 8 (e.g., skewness values, correlation coefficients, explained variance percentages) is tied to a specific computed statistic rather than a visual impression alone, to avoid overstating or misreading a pattern.
Documentation	The dataset's source, generation method, variable dictionary, missing-data handling, and all algorithm parameters (random seed, t-SNE perplexity, PCA components) are documented in Sections 3 and 5 so the analysis can be reproduced exactly.
Responsible Analysis	Conclusions are scoped to what the data supports: for example, the report explicitly notes where PCA and t-SNE do not align with ProductCategory rather than forcing an interpretation that the data does not support.

10. Conclusion

This assignment worked through a 1,500-record synthetic retail sales and customer behaviour dataset spanning 8 product categories, 5 regions and 3 customer segments. I implemented and compared amount, distribution, proportion, relationship and correlation visualization techniques, then followed up with dimensionality reduction using PCA and t-SNE. The key patterns that emerged: a strongly right-skewed SalesAmount distribution (skewness ~ 2.7) driven by a long tail of high-value orders; a strong positive correlation between UnitPrice and SalesAmount and a moderate one between Quantity and SalesAmount; a negligible relationship between CustomerRating and either order value or price; and PCA/t-SNE embeddings that reveal structure driven by pricing and quantity rather than product category, since category was never one of the numerical features to begin with.

CO1 concepts - visualization fundamentals, aesthetic mappings, scales and colour - run throughout Section 5.1 and show up in every figure's design choices afterward. CO2 concepts (amount, distribution and proportion visualization) are covered in Sections 5.2-5.4, with each module going beyond the minimum required number of techniques and comparing at least two alternatives. CO3 concepts (relationship, correlation and dimensionality reduction) come through in Sections 5.5-5.7, wrapping up with a direct, evidence-based comparison of PCA and t-SNE.

One thing that came through clearly across every comparison: no single visualization technique is universally best. The right choice always depends on the number of dimensions involved, how much precision is needed, and whether the goal is exact comparison or pattern-spotting. Key decisions - preferring heatmaps and stacked bars once a second grouping variable shows up, or reaching for violin plots when distribution shape (not just central tendency) is what matters - were all justified against the actual generated figures rather than general assumptions.

This work does have its limits. The dataset is synthetic rather than real-world, so it probably understates some of the noisier, messier relationships you'd find in genuine retail data. The t-SNE embedding was computed on a 600-record subsample for the sake of computational tractability, so it may not fully represent the whole dataset. And PCA/t-SNE were applied only to the six numerical variables, leaving how the categorical variables interact with the reduced space unexplored. Given more time, I'd want to one-hot encode the categorical variables before

dimensionality reduction, try UMAP as a third comparison technique, and validate the whole pipeline against a real, larger-scale retail dataset like Kaggle's Superstore or Online Retail data.

11. Student Reflection

1. What was learned beyond the classroom

Beyond what the course covered on chart types, the big lesson for me was realizing that good visualization design is really about matching the encoding to the specific question being asked, not just to the variable type. Building the whole pipeline end-to-end - from generating and documenting the dataset all the way through comparing PCA and t-SNE - also gave me a much clearer sense of how techniques that get taught individually actually fit together into one analytical workflow. That's a very different feeling from seeing each chart type in isolation during lectures.

2. Most challenging visualization technique and why

The dimensionality reduction stage, t-SNE especially, gave me the most trouble. Unlike a bar chart or histogram, there's no fixed, obvious way to read its output - the axes don't carry any direct meaning, distances between clusters aren't reliable, and the whole result shifts depending on the perplexity parameter and random seed. Figuring out what t-SNE was actually telling me about the data, and what it wasn't, took a lot more careful reading and experimenting than any of the standard chart types did.

3. A specific problem faced selecting a visualization and how it was solved

When I first tackled the region-by-segment proportion breakdown, my instinct was to draw a separate pie chart for each region - but with 5 regions and 3 segments, that meant five different charts that were hard to compare against each other. I fixed this by switching to a single stacked, 100%-normalized bar chart instead, which puts every region on the same aligned baseline and lets the segment proportions be compared directly across regions in one figure instead of five.

4. How understanding of data types influenced visualization choices

Recognizing which variables were continuous (SalesAmount, UnitPrice), which were discrete (Quantity, PurchaseFrequency), and which were nominal categorical (ProductCategory, Region) directly decided which chart families were even on the table - a KDE or Q-Q plot only makes sense for a continuous variable, for instance, while a stacked bar or pie chart needs a categorical grouping variable. That classification step, done early in Section 5.1, ended up shaping almost every visualization decision later in the report.

5. Which visualization gave the most useful insight, and why

The Pearson correlation matrix heatmap (Figure 15) was, hands down, the single most useful visualization in this whole project. It condensed fifteen pairwise relationships into one view and immediately told me which pairs (UnitPrice-SalesAmount, Quantity-SalesAmount) were worth digging into further with scatter or hexbin plots, and which ones (Discount, CustomerRating) weren't strongly related to anything else - saving me the time it would've taken to inspect every pair one by one.

6. How dimensionality reduction changed understanding of multivariate data

Before I ran PCA and t-SNE, I assumed six correlated numerical variables would collapse into two or three visually obvious groupings tied to product category. Instead, both techniques showed that most of the variance is driven by pricing and quantity patterns that cut across categories, not category membership itself. That was a

useful correction to my intuition, and a good reminder that dimensionality reduction is a genuinely exploratory technique - not a way to confirm something you already believed going in.

7. What would be improved with more time or resources

Given more time, I'd validate the pipeline against a real-world retail dataset instead of a synthetic one, one-hot encode the categorical variables so PCA and t-SNE could use them directly, run t-SNE on the full dataset instead of a subsample (or swap in a faster method like UMAP for large-scale use), and build an interactive dashboard - Plotly or Streamlit, say - so a retail analyst could filter by region or category and watch the visualizations update live instead of working from a fixed set of static figures.

12. References

- [1] Kaggle, "Superstore Sales Dataset," [Online]. Available: <https://www.kaggle.com/datasets/vivek468/superstore-dataset-final> (referenced as the structural basis for the synthetic dataset used in this report).
- [2] The pandas development team, "pandas documentation," [Online]. Available: <https://pandas.pydata.org/docs/>
- [3] Harris, C.R. et al., "Array programming with NumPy," Nature, 585, 357-362 (2020). NumPy documentation: <https://numpy.org/doc/>
- [4] Hunter, J.D., "Matplotlib: A 2D Graphics Environment," Computing in Science & Engineering, vol. 9, no. 3, 2007. Documentation: <https://matplotlib.org/stable/>
- [5] Waskom, M.L., "seaborn: statistical data visualization," Journal of Open Source Software, 6(60), 3021, 2021. Documentation: <https://seaborn.pydata.org/>
- [6] Pedregosa, F. et al., "Scikit-learn: Machine Learning in Python," JMLR 12, pp. 2825-2830, 2011. Documentation: <https://scikit-learn.org/stable/documentation.html>
- [7] Van der Maaten, L. and Hinton, G., "Visualizing Data using t-SNE," Journal of Machine Learning Research, 9(Nov), 2579-2605, 2008.
- [8] Virtanen, P. et al., "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python," Nature Methods, 17, 261-272, 2020.
- [9] Munzner, T., "Visualization Analysis and Design," CRC Press, 2014 (course reference text on visual mappings, scales and design justification).
- [10] DSA06 - Data Handling and Visualization, course lecture material, Saveetha School of Engineering (SIMATS).

Appendix: Full Python Source Code

The complete Python source code used to generate the dataset and all visualizations/analyses in this report is listed below, organized by module, so it can be run end-to-end.

A.1 Dataset Generation

```
import numpy as np, pandas as pd
rng = np.random.default_rng(42)
N = 1500

categories = ["Electronics", "Furniture", "Clothing", "Groceries", "Sports", "Home Decor", "Toys", "Books"]
cat_p = [0.16, 0.14, 0.18, 0.15, 0.10, 0.09, 0.08, 0.10]
regions = ["North", "South", "East", "West", "Central"]
region_p = [0.22, 0.24, 0.18, 0.20, 0.16]
segments = ["Consumer", "Corporate", "Home Office"]
seg_p = [0.55, 0.28, 0.17]

category = rng.choice(categories, N, p=cat_p)
region = rng.choice(regions, N, p=region_p)
segment = rng.choice(segments, N, p=seg_p)

base_price = {"Electronics": 180, "Furniture": 220, "Clothing": 45, "Groceries": 18,
              "Sports": 60, "Home Decor": 40, "Toys": 30, "Books": 15}
price = np.array([max(2, rng.gamma(4, base_price[c]/4)) for c in category])
quantity = rng.poisson(3, N) + 1
discount = np.round(rng.beta(2, 6, N) * 0.5, 2)
sales_amount = np.round(price * quantity * (1 - discount), 2)
rating = np.clip(rng.normal(3.9, 0.9, N), 1, 5).round(1)
purchase_frequency = rng.poisson(4, N) + 1

start = pd.Timestamp("2024-01-01")
order_date = start + pd.to_timedelta(rng.integers(0, 730, N), unit="D")

product_names = {c: [f"{c} Item {i}" for i in range(1, 21)] for c in categories}
product_name = [rng.choice(product_names[c]) for c in category]

df = pd.DataFrame({
    "OrderID": [f"ORD{100000+i}" for i in range(N)],
    "OrderDate": order_date,
    "ProductCategory": category,
    "ProductName": product_name,
    "Region": region,
    "CustomerSegment": segment,
    "Quantity": quantity,
    "UnitPrice": price.round(2),
    "Discount": discount,
    "SalesAmount": sales_amount,
    "CustomerRating": rating,
    "PurchaseFrequency": purchase_frequency,
})

# inject a few missing values
for col in ["CustomerRating", "Discount"]:
    idx = rng.choice(N, size=int(N*0.02), replace=False)
    df.loc[idx, col] = np.nan

df.to_csv("/home/claude/proj/retail_data.csv", index=False)
print(df.shape)
print(df.dtypes)
print(df.isna().sum())
```

A.2 Visualization and Analysis Pipeline

```

import pandas as pd, numpy as np, matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats as spstats
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
import time, json, os

sns.set_theme(style="whitegrid")
IMG = "/home/claude/proj/imgs"
os.makedirs(IMG, exist_ok=True)
df = pd.read_csv("/home/claude/proj/retail_data.csv", parse_dates=["OrderDate"])
df["Discount"] = df["Discount"].fillna(df["Discount"].median())
df["CustomerRating"] = df["CustomerRating"].fillna(df["CustomerRating"].median())

results = {}

def save(fig, name):
    fig.savefig(f"{IMG}/{name}.png", dpi=150, bbox_inches="tight")
    plt.close(fig)

palette = sns.color_palette("Set2", 8)
cat_order = df.groupby("ProductCategory")["SalesAmount"].sum().sort_values(ascending=False).index

# 1. Bar plot - sales by category
fig, ax = plt.subplots(figsize=(7,4.5))
s = df.groupby("ProductCategory")["SalesAmount"].sum().reindex(cat_order)
ax.bar(s.index, s.values, color=palette)
ax.set_title("Total Sales Amount by Product Category")
ax.set_xlabel("Product Category"); ax.set_ylabel("Total Sales Amount ($)")
plt.xticks(rotation=30, ha="right")
save(fig, "fig01_bar_sales_category")

# 2. Grouped bar - regional sales by segment
fig, ax = plt.subplots(figsize=(7.5,4.5))
piv = df.groupby(["Region", "CustomerSegment"])["SalesAmount"].sum().unstack()
piv.plot(kind="bar", ax=ax, color=sns.color_palette("Set2",3))
ax.set_title("Regional Sales by Customer Segment (Grouped Bar)")
ax.set_xlabel("Region"); ax.set_ylabel("Total Sales Amount ($)")
ax.legend(title="Segment")
plt.xticks(rotation=0)
save(fig, "fig02_grouped_bar_region_segment")

# 3. Heatmap - product performance (category x region avg sales)
fig, ax = plt.subplots(figsize=(7,4.5))
hp = df.pivot_table(index="ProductCategory", columns="Region", values="SalesAmount", aggfunc="mean")
sns.heatmap(hp, annot=True, fmt=".0f", cmap="YlGnBu", ax=ax, cbar_kws={"label": "Avg Sales Amount ($)"})
ax.set_title("Average Sales Amount: Category x Region (Heatmap)")
save(fig, "fig03_heatmap_category_region")

# 4. Histogram
fig, ax = plt.subplots(figsize=(6.5,4.2))
ax.hist(df["SalesAmount"], bins=30, color=palette[0], edgecolor="white")
ax.set_title("Distribution of Sales Amount (Histogram)")
ax.set_xlabel("Sales Amount ($)"); ax.set_ylabel("Frequency")
save(fig, "fig04_histogram_sales")

# 5. KDE
fig, ax = plt.subplots(figsize=(6.5,4.2))
sns.kdeplot(df["SalesAmount"], fill=True, color=palette[1], ax=ax)

```

```

ax.set_title("Density Plot (KDE) of Sales Amount")
ax.set_xlabel("Sales Amount ($)"); ax.set_ylabel("Density")
save(fig, "fig05_kde_sales")

# 6. Boxplot
fig, ax = plt.subplots(figsize=(7.5,4.5))
sns.boxplot(data=df, x="ProductCategory", y="SalesAmount", hue="ProductCategory", palette="Set2", ax=ax, legend=False)
ax.set_title("Sales Amount by Category (Boxplot)")
plt.xticks(rotation=30, ha="right")
save(fig, "fig06_boxplot_sales_category")

# 7. Violin
fig, ax = plt.subplots(figsize=(7.5,4.5))
sns.violinplot(data=df, x="ProductCategory", y="SalesAmount", hue="ProductCategory", palette="Set2", ax=ax, legend=False)
ax.set_title("Sales Amount by Category (Violin Plot)")
plt.xticks(rotation=30, ha="right")
save(fig, "fig07_violin_sales_category")

# 8. ECDF
fig, ax = plt.subplots(figsize=(6.5,4.2))
sns.ecdfplot(df["SalesAmount"], ax=ax, color=palette[2])
ax.set_title("ECDF of Sales Amount")
ax.set_xlabel("Sales Amount ($)"); ax.set_ylabel("Cumulative Proportion")
save(fig, "fig08_ecdf_sales")

# 9. Q-Q plot
fig, ax = plt.subplots(figsize=(5.5,5.5))
spstats.probplot(df["SalesAmount"], dist="norm", plot=ax)
ax.set_title("Q-Q Plot of Sales Amount (Normality Check)")
save(fig, "fig09_qqplot_sales")

skew = spstats.skew(df["SalesAmount"]); kurt = spstats.kurtosis(df["SalesAmount"])
results["skewness"] = round(float(skew),3)
results["kurtosis"] = round(float(kurt),3)

# 10. Pie chart
fig, ax = plt.subplots(figsize=(6,6))
s2 = df.groupby("ProductCategory")["SalesAmount"].sum().reindex(cat_order)
ax.pie(s2.values, labels=s2.index, autopct="%1.1f%%", colors=palette, startangle=90)
ax.set_title("Product Category Contribution to Total Sales (Pie Chart)")
save(fig, "fig10_pie_category")

# 11. Stacked bar - region contribution by segment (proportion)
fig, ax = plt.subplots(figsize=(7,4.5))
piv2 = df.pivot_table(index="Region", columns="CustomerSegment", values="SalesAmount", aggfunc="sum")
piv2_pct = piv2.div(piv2.sum(axis=1), axis=0)*100
piv2_pct.plot(kind="bar", stacked=True, ax=ax, color=sns.color_palette("Set2",3))
ax.set_title("Regional Sales Composition by Segment (Stacked Bar, %)")
ax.set_xlabel("Region"); ax.set_ylabel("Percentage of Sales (%)")
ax.legend(title="Segment", bbox_to_anchor=(1.02,1), loc="upper left")
plt.xticks(rotation=0)
save(fig, "fig11_stacked_bar_region_segment")

# 12. Scatter - quantity vs sales
fig, ax = plt.subplots(figsize=(6.5,4.5))
ax.scatter(df["Quantity"], df["SalesAmount"], alpha=0.4, s=18, color=palette[3])
ax.set_title("Quantity vs Sales Amount (Scatterplot)")
ax.set_xlabel("Quantity"); ax.set_ylabel("Sales Amount ($)")
save(fig, "fig12_scatter_qty_sales")

# 13. Bubble chart - price vs quantity, bubble=rating
fig, ax = plt.subplots(figsize=(7,4.8))
samp = df.sample(300, random_state=1)
sc = ax.scatter(samp["UnitPrice"], samp["Quantity"], s=samp["CustomerRating"]*40, alpha=0.5, c=samp["CustomerRating"],

```

```

cmap="viridis")
ax.set_title("Price vs Quantity (Bubble Size/Color = Customer Rating)")
ax.set_xlabel("Unit Price ($)"); ax.set_ylabel("Quantity")
plt.colorbar(sc, ax=ax, label="Customer Rating")
save(fig, "fig13_bubble_price_qty_rating")

# 14. Hexbin - discount vs sales
fig, ax = plt.subplots(figsize=(6.5,4.8))
hb = ax.hexbin(df["Discount"], df["SalesAmount"], gridsize=25, cmap="Blues")
ax.set_title("Discount vs Sales Amount (Hexbin Plot)")
ax.set_xlabel("Discount"); ax.set_ylabel("Sales Amount ($)")
plt.colorbar(hb, ax=ax, label="Count")
save(fig, "fig14_hexbin_discount_sales")

# 15. Correlation matrix heatmap
numcols = ["Quantity", "UnitPrice", "Discount", "SalesAmount", "CustomerRating", "PurchaseFrequency"]
corr = df[numcols].corr(method="pearson")
fig, ax = plt.subplots(figsize=(6.5,5.5))
sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm", center=0, ax=ax, square=True)
ax.set_title("Pearson Correlation Matrix (Numerical Variables)")
save(fig, "fig15_correlation_matrix")
results["corr"] = corr.round(3).to_dict()

# find strong/weak pairs
pairs = []
for i,a in enumerate(numcols):
    for b in numcols[i+1:]:
        pairs.append((a,b, corr.loc[a,b]))
pairs.sort(key=lambda x: abs(x[2]), reverse=True)
results["top_corr_pairs"] = [(a,b,round(float(v),3)) for a,b,v in pairs[:3]]
results["weak_corr_pairs"] = [(a,b,round(float(v),3)) for a,b,v in sorted(pairs, key=lambda x: abs(x[2]))[:3]]

# --- Dimensionality reduction ---
X = df[numcols].copy()
scaler = StandardScaler()
Xs = scaler.fit_transform(X)

pca = PCA(n_components=min(6, Xs.shape[1]))
pcs = pca.fit_transform(Xs)
evr = pca.explained_variance_ratio_
results["pca_explained_variance_ratio"] = [round(float(v),4) for v in evr]

# scree plot
fig, ax = plt.subplots(figsize=(6.5,4.2))
ax.plot(range(1,len(evr)+1), evr, marker="o", color=palette[4])
ax.set_title("PCA Scree Plot (Explained Variance Ratio)")
ax.set_xlabel("Principal Component"); ax.set_ylabel("Explained Variance Ratio")
save(fig, "fig16_pca_scree")

cats_codes = pd.Categorical(df["ProductCategory"], categories=cat_order)
colors_map = dict(zip(cat_order, palette))
point_colors = df["ProductCategory"].map(colors_map)

fig, ax = plt.subplots(figsize=(7,5.5))
for c in cat_order:
    m = df["ProductCategory"]==c
    ax.scatter(pcs[m.values,0], pcs[m.values,1], s=14, alpha=0.6, label=c, color=colors_map[c])
ax.set_title("PCA: First Two Principal Components (Colored by Category)")
ax.set_xlabel(f"PC1 ({evr[0]*100:.1f}% variance)"); ax.set_ylabel(f"PC2 ({evr[1]*100:.1f}% variance)")
ax.legend(title="Category", bbox_to_anchor=(1.02,1), loc="upper left", fontsize=8)
save(fig, "fig17_pca_2d_scatter")

# t-SNE (on a subsample for speed)
tsne_sub = df.sample(600, random_state=1)

```

```

Xs_sub = scaler.transform(tsne_sub[numcols])
tsne = TSNE(n_components=2, perplexity=30, random_state=42, init="pca")
te = tsne.fit_transform(Xs_sub)

fig, ax = plt.subplots(figsize=(7,5.5))
for c in cat_order:
    m = tsne_sub["ProductCategory"]==c
    ax.scatter(te[m.values,0], te[m.values,1], s=14, alpha=0.6, label=c, color=colors_map[c])
ax.set_title("t-SNE: 2D Embedding (Colored by Category)")
ax.set_xlabel("t-SNE Dimension 1"); ax.set_ylabel("t-SNE Dimension 2")
ax.legend(title="Category", bbox_to_anchor=(1.02,1), loc="upper left", fontsize=8)
save(fig, "fig18_tsne_2d_scatter")

# side by side PCA vs tSNE
fig, axes = plt.subplots(1,2, figsize=(12,5.2))
for c in cat_order:
    m = df["ProductCategory"]==c
    axes[0].scatter(pcs[m.values,0], pcs[m.values,1], s=10, alpha=0.6, label=c, color=colors_map[c])
axes[0].set_title("PCA (PC1 vs PC2)"); axes[0].set_xlabel("PC1"); axes[0].set_ylabel("PC2")
for c in cat_order:
    m = tsne_sub["ProductCategory"]==c
    axes[1].scatter(te[m.values,0], te[m.values,1], s=10, alpha=0.6, label=c, color=colors_map[c])
axes[1].set_title("t-SNE"); axes[1].set_xlabel("Dim 1"); axes[1].set_ylabel("Dim 2")
axes[1].legend(title="Category", bbox_to_anchor=(1.02,1), loc="upper left", fontsize=8)
fig.suptitle("Side-by-Side Comparison: PCA vs t-SNE")
save(fig, "fig19_pca_vs_tsne")

# --- Performance benchmarking ---
def bench(n):
    sub = df.sample(n, replace=n>len(df), random_state=1).reset_index(drop=True)
    timings = {}
    t0=time.time(); _=pd.read_csv("/home/claude/proj/retail_data.csv").sample(n, replace=n>len(df), random_state=1);
    timings["load"]=time.time()-t0
    t0=time.time()
    f,a = plt.subplots(); a.bar(sub.groupby("ProductCategory")["SalesAmount"].sum().index,
    sub.groupby("ProductCategory")["SalesAmount"].sum().values); plt.close(f)
    timings["amount_viz"]=time.time()-t0
    t0=time.time()
    f,a = plt.subplots(); a.hist(sub["SalesAmount"], bins=30); plt.close(f)
    timings["distribution_viz"]=time.time()-t0
    t0=time.time(); _=sub[numcols].corr(); timings["correlation"]=time.time()-t0
    t0=time.time()
    Xs_n = StandardScaler().fit_transform(sub[numcols]); PCA(n_components=2).fit_transform(Xs_n)
    timings["pca"]=time.time()-t0
    t0=time.time()
    TSNE(n_components=2, perplexity=min(30,max(5,n//20)), random_state=42, init="pca").fit_transform(Xs_n[:min(n,500)])
    timings["tsne"]=time.time()-t0
    return timings

perf = {}
for n in [1000,5000,10000]:
    perf[n] = bench(n)
results["performance"] = perf

# performance line chart
fig, ax = plt.subplots(figsize=(7,5.5))
metrics = ["load","amount_viz","distribution_viz","correlation","pca","tsne"]
xs = [1000,5000,10000]
for m,c in zip(metrics, sns.color_palette("tab10",6)):
    ax.plot(xs, [perf[n][m] for n in xs], marker="o", label=m, color=c)
ax.set_title("Execution Time vs Dataset Size")
ax.set_xlabel("Number of Records"); ax.set_ylabel("Execution Time (seconds)")
ax.legend(fontsize=8)
save(fig, "fig21_performance_line_chart")

```

```
# summary stats table data
results["describe"] = df[numcols].describe().round(2).to_dict()
results["shape"] = df.shape
results["missing"] = df.isna().sum().to_dict()

with open("/home/claude/proj/results.json","w") as f:
    json.dump(results, f, indent=2, default=str)

print("DONE")
print(json.dumps({k:v for k,v in results.items() if k not in ["describe","corr"]}, indent=2, default=str))
```

A.3 Workflow Diagram Generation

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from matplotlib.patches import FancyBboxPatch, FancyArrowPatch

steps = [
    "Raw Dataset",
    "Data Loading",
    "Data Inspection",
    "Data Type Identification",
    "Visual Mapping & Selection",
    "Amount / Distribution / Proportion \nRelationship / Correlation / Dim. Reduction",
    "Visualization Comparison",
    "Results and Interpretation",
    "Final Analytical Insights",
]

fig, ax = plt.subplots(figsize=(6.5, 11))
ax.set_xlim(0, 10)
ax.set_ylim(0, len(steps)*1.2 + 1)
ax.axis("off")

box_w, box_h = 8.5, 0.9
colors = plt.cm.Blues([0.35 + 0.06*i for i in range(len(steps))])

ys = []
for i, step in enumerate(steps):
    y = (len(steps)-1-i)*1.2 + 0.5
    ys.append(y)
    box = FancyBboxPatch((0.75, y), box_w, box_h, boxstyle="round,pad=0.08,rounding_size=0.12",
        linewidth=1.2, edgecolor="#2c5f8a", facecolor=colors[i])
    ax.add_patch(box)
    ax.text(5, y+box_h/2, step, ha="center", va="center", fontsize=9.5, wrap=True, color="#1a1a1a")

for i in range(len(steps)-1):
    y_top = ys[i]
    y_bot = ys[i+1] + box_h
    arrow = FancyArrowPatch((5, y_top), (5, y_bot), arrowstyle="->", mutation_scale=15, color="#2c5f8a", linewidth=1.4)
    ax.add_patch(arrow)

ax.set_title("Overall Solution Workflow", fontsize=13, fontweight="bold", pad=15)
plt.tight_layout()
plt.savefig("/home/claude/proj/imgs/fig20_workflow_diagram.png", dpi=150, bbox_inches="tight")
print("saved")
```